Power Lines: batch size, weight decay and long-horizon training

AdamW timescales, optimal versus critical batch and a replication protocol

Why this paper matters for our overtrained model

Bergsma et al., Power Lines v2 studies AdamW weight decay and batch scaling across model size and training horizon, including heavily overtrained settings up to1,280 TPP. It makes a LR/batch-only tuning plan incomplete: weight decay interacts with both. Primary artifacts are the PDF and LaTeX/code metadata.

The reported experiments use Cerebras CS-3, SlimPajama, a GPT-like ALiBi/SwiGLU family, AdamW/μP and a fixed1.1B-token validation set. The schedule is linear decay with 10% warmup. This is close to our **overtraining regime**, not our exact hardware, model, corpus or schedule.

1. Derive the timescale before applying its fit

Ignoring the gradient term momentarily, decoupled AdamW decay contributes

$$W_{s+1} \approx (1 - \eta\lambda)W_s.$$

At approximately constant peak LR η and decay λ, its update-count timescale is $1/(\eta\lambda)$. With $S = D/B_t$ updates over horizon D and token batch B_t, the normalized timescale is

$$\tau = \frac{1/(\eta\lambda)}{D/B_t} = \frac{B_t}{\eta\lambda D}.$$

During annealing the instantaneous value changes. The paper evaluates the nominal timescale at peak LR under matched schedule shapes; it is not claiming the LR is actually constant for the whole run. Group-wise actual LR/decay values matter under μP. A base LR printed in a config may not be the LR used by each hidden weight.

2. What the weight-decay sweeps find

At fixed N, D, increasing batch while preserving the tuned timescale can require changing decay. The paper then finds that optimal normalized timescale varies with tokens per parameter:

$$\tau_\star \approx 1.084(D/N)^{-0.527}, \qquad \lambda_\star = \frac{B_t}{\eta D \tau_\star}.$$

This is empirical across the studied family. It corrects an unconditional "keep τ constant as everything scales" rule. It also gives a practical alternative to repeatedly changing LR when horizon or batch changes: tune decay/timescale on small cells, fit its TPP dependence, and validate at larger scale.

Concrete evidence (§2.4): a 111M/200TPP example obtains held-out losses 2.810 with default HPs, 2.808 after LR tuning and 2.805 after decay tuning. Table 1 compares tuning decay versus LR at 610M/20TPP across batches; decay tuning is superior in six of eight comparisons. These are small but controlled differences, not proof of a universal decay optimum.

The paper reports a strong timescale power-law fit and additional held-out scale checks. That supports testing the prescription in our setup. It does not justify inserting its decay value into MuonH: Hyperball matrices use a different norm constraint, and some scalar groups may still have separate decay.

3. Optimal and critical batch answer different questions

Loss-optimal batch $B_\star(N, D)$ minimizes final loss for fixed model and token horizon. **Critical batch** describes diminishing returns from increasing data parallelism at a specified target loss.

If $D(B)$ tokens and $S(B)$ updates reach target loss, the paper uses the McCandlish-style tradeoff

$$\left(\frac{S}{S_{\min}} - 1\right)\left(\frac{D}{D_{\min}} - 1\right) = 1, \qquad B_{\text{crit}} = D_{\min}/S_{\min}.$$

Since $B = D/S$, this implies $D = D_{\min}(1 + B/B_{\text{crit}})$ and $S = S_{\min}(1 + B_{\text{crit}}/B)$ under this fitted relation. Small batches need many updates but little extra data; large batches reduce steps with diminishing returns and increasing token demand. This mathematical interpretation does not by itself determine fastest wall time on a specific cluster.

4. The coefficient units are easy to get wrong

Figure 1 reports

$$B_{\star,s} = 0.0306 D^{0.383}, \qquad B_{\text{crit},s} = 0.0471 D_{\min}^{0.462}.$$

The outputs here are sequences of length 2,048; D **and** $D_{\min}$ **are tokens.** Multiply by2,048 for token batches. In the timescale equation, use B_t in tokens. AppendixE explicitly distinguishes these conventions.

The exponents imply that quadrupling horizon raises fitted loss-optimal sequence batch by about 1.70; the corresponding critical-batch expression grows by about 1.90 when its $D_{\min}$ increases fourfold. These are consequences of the fitted equations, not new experiment observations.

Changing context length means rechecking packing/attention cost and the relevant batch conventions; a sequence-batch coefficient is not portable by changing the sequence-length field alone. Likewise, a model-size label that includes embeddings in our plan can differ from the paper's parameter convention.

5. µP does not remove horizon and decay tuning

µP aims to preserve useful width transfer through coordinated initialization, parameter-group LR and other rules. The paper distinguishes base LR from actual scaled LR. Its results show why a width-transferred LR is not the entire training recipe: batch and decay still interact with token horizon.

Do not summarize µP as "divide every LR by width." Implement and inspect each group. Depth changes, MoE routing and expert-bank normalization need their own tests. Marin's AdamH/MuonH prescriptions differ from AdamW/µP, so compare fully tuned algorithms instead of combining their coefficients.

Our replication and adaptation protocol

1. Recreate a small standard dense or transparent MoE reference with documented μP/group semantics, tokenizer, corpus and schedule. For a faithful paper replication, match its family/data and context before extending it.
2. At a reference (N, D), sweep several feasible token batches and decay values while holding the peak-LR prescription fixed. Check instability and whether minima are bracketed.
3. Repeat at independent horizons near the intended overtraining range, including 200+TPP under a declared count convention. Convert every trial to τ using actual LR/decay and token batch.
4. Fit $\log \tau = a + b \log(D/N)$, then validate decay predictions at unseen size/horizon with local LR and decay brackets. Compare with a constant-decay baseline given equal tuning effort.
5. To estimate critical batch, train enough trajectories across batches to interpolate tokens/updates required for common **held-out target losses**. Do not call the fastest throughput setting a statistical critical batch.
6. Profile wall time versus token batch and accumulation on our actual GPU topology. Choose on a quality/runtime frontier, not a coefficient alone.
7. Add cyber data only after a baseline is qualified; run selected transfer cells before deciding whether new coefficients are needed.

For repeated data, record unique bucket pools and cumulative epochs separately from D. A long-horizon law fit on diverse fresh text may not describe memorizing a tiny repeated corpus. For our MoE candidate, total stored TPP, active TPP and executed compute each need explicit reporting.

The Marin tuning guide integrates these checks into a staged grid/adaptive search. The paper supplies evidence for including decay/timescale; it does not select our hero optimizer or promise a universal long-horizon coefficient.